

Scope and Objectives

Define clear scope and success metrics up front. Decide if the first version is strictly read-only (safe analytics) or if write capabilities (INSERT/UPDATE) are needed later. Pick initial target data sources (e.g. a single Postgres DB) before scaling to multiple DBs, data warehouses, and data lakes. Establish trust boundaries (e.g. user authentication, allowed schemas) and success criteria: execution accuracy, exact-match vs. execution match, clarification rate, self-correction rate, latency, token cost, and safety violations. Recent surveys stress that text-to-SQL success must be measured on many dimensions (Spider 2.0, BIRD, etc.) ¹ ². Plan to iterate on metrics as the agent evolves.

LangGraph Agent Architecture (State, Nodes, Edges)

Design the solution as a **LangGraph agent graph**. Use a shared `State` object containing fields like `user_query`, `schema`, `retrieved_values`, `dialect`, `candidate_sql`, `result`, `error`, and `clarification_needed`. Define LangGraph **nodes** for each step: intent classification, ambiguity detection, schema retrieval, value/content retrieval, SQL generation, SQL validation, SQL execution, error correction, and answer synthesis. Use **conditional edges** to branch (e.g. if `state.needs_clarification` go to a clarification node; if execution fails go to correction node; else return result). LangGraph explicitly supports complex, stateful agent flows ³ ⁴. For example:

```
from langgraph import Graph
from langgraph.types import interrupt, Command

# Define the shared State data structure for our agent
class State:
    user_query: str = ""
    schema: dict = None
    values: dict = None
    dialect: str = ""
    sql: str = ""
    result: any = None
    error: str = ""
    needs_clarification: bool = False

# Example node: classify intent or detect ambiguity
def classify_intent(state: State) -> State:
    # (Optional) set needs_clarification if query is vague
    if "top customers" in state.user_query.lower():
        state.needs_clarification = True
    return state
```

```
# Build the LangGraph
graph = Graph[State]()
graph.add_node("intent_node", classify_intent)
# ... add other nodes similarly ...
```

Use LangGraph's `interrupt()` to pause for human input (HITL) when needed. For instance, if `state.needs_clarification` is true, send an interrupt to ask the user for more detail, then resume when input is provided ⁵. Internally, LangGraph saves the state on interrupt so execution can continue later ⁵.

Human-in-the-Loop Clarification

Do **not** let the model guess when the user's request is under-specified. Instead, include a dedicated *clarification node* using LangGraph interrupts ⁵. For example, if the query is "Show top customers", the agent should interrupt and ask follow-up questions (e.g. "Top by what metric? sales amount or count?"). Here's an example of a clarification node in code:

```
def clarification_node(state: State) -> State:
    # Pause execution and ask the user to clarify the metric or time window
    clarified = interrupt("Which metric defines 'top customers' (e.g. total
revenue, order count)?")
    state.user_query += f" TOP BY {clarified}"
    return state

graph.add_node("clarification_node", clarification_node)
graph.add_edge("intent_node", "clarification_node", condition=lambda s:
s.needs_clarification)
```

This leverages LangGraph's streaming and checkpointing: execution will pause, store the state, and await user input ⁵. After resuming, the graph continues to later nodes. Human approval steps can also guard risky actions (e.g. before executing a generated SQL) by using `interrupt()` ⁵.

Metadata Ingestion and Universal Connectivity

Instead of feeding raw DDL strings to the model, build a robust **metadata layer**. Ingest table schemas, column types, primary/foreign keys, comments, and samples. Also implement **content retrieval** so the agent can fetch actual cell values (e.g. sample countries or dates) to avoid guesswork in WHERE clauses ⁶. Treat this as a retrieval problem: e.g. cache "SELECT DISTINCT state FROM customers LIMIT 5" so the agent knows exact string formats (e.g. "TX" vs "Texas"). Recent work emphasizes schema linking and value retrieval as core tasks in text-to-SQL ⁶ ⁷.

For **connectivity**, create a **connector registry** rather than a one-size driver. For each target (Postgres, MySQL, Snowflake, etc.), register a connector class (e.g. using SQLAlchemy engines) that knows how to connect and translate queries. For example:

```
from sqlalchemy import create_engine

class SQLConnector:
    def __init__(self, uri: str):
        # Create a SQLAlchemy Engine for the given URI
        self.engine = create_engine(uri)

    def get_schema(self):
        # Use SQLAlchemy inspection to reflect tables/columns, PK/FK etc.
        meta = MetaData()
        meta.reflect(bind=self.engine)
        return meta

    def execute(self, sql: str):
        # Execute the SQL in a safe, read-only context
        with self.engine.connect() as conn:
            return conn.execute(sql).fetchall()

# Example registry mapping (dialect or key -> connector instance)
connectors = {
    "postgres": SQLConnector("postgresql+psycopg2://user:pass@host/db"),
    "snowflake": SQLConnector("snowflake://user:pass@account/db/schema"),
    # Add other connectors as needed
}
```

The SQLAlchemy `Engine` is the “home base” for a database connection and its *Dialect* (protocol) ⁸ ⁹ . Using SQLAlchemy lets us leverage built-in dialects and connection pooling. Store secrets/configs securely, and use per-source checkpoints or tokens to resume as needed.

For **data lakes/lakehouses**, route queries through a compute layer. Raw S3 or HDFS isn’t directly queried by SQLAlchemy, so integrate engines like AWS Athena (serverless SQL on S3) or Presto/Trino. Athena is a serverless interactive SQL service over S3 ¹⁰ . Trino is a fast distributed SQL query engine for big data, supporting ANSI SQL and federated queries across many systems ¹¹ ¹² . For example, you might use PyAthena or trino-python-client under the hood to route queries. The key is *dialect-aware routing*:

- If the target is Snowflake, use its SQL flavor.
- If the target is Athena, use Presto SQL syntax.
- If it’s BigQuery, use BigQuery SQL.

Detect the target dialect early (e.g. by config or metadata) and inject dialect-specific rules or function mappings into prompts. Many text-to-SQL surveys stress that enterprise tasks involve *multiple dialects* and systems ¹³ , so this step is crucial.

Dialect Handling and Translation

Implement a **SQL dialect translation layer**. The agent can generate SQL in a canonical form (e.g. a generic SQL), then translate or adjust it per target engine. For example, date truncation, LIMIT syntax, quoting styles differ (Snowflake uses double-quotes, BigQuery backticks, etc.). Maintain a map of dialect differences:

```
dialect_rules = {
    "postgres": {"limit_syntax": "LIMIT {n}", "identifier_quote": "'"},
    "snowflake": {"limit_syntax": "LIMIT {n}", "identifier_quote": '"'},
    "bigquery": {"limit_syntax": "LIMIT {n}", "identifier_quote": "`"},
    # etc.
}

def translate_sql(generic_sql: str, target: str) -> str:
    rules = dialect_rules[target]
    # Example: replace generic LIMIT with dialect-specific
    return generic_sql.format(limit_syntax=rules["limit_syntax"])
```

In prompts, explicitly tell the LLM which dialect to use (e.g. “Use Snowflake SQL syntax”). This avoids generating invalid functions or syntax. Some advanced systems generate in one form then post-process. Spider 2.0 tasks often span multiple dialects in long workflows ¹³, so robust dialect support is needed.

Semantic Layer and Ontology Mapping

Build a **semantic/ontology layer** on top of raw schemas. Define mappings from high-level business concepts (“active customer”, “churn rate”, “revenue-at-risk”) to actual tables/columns or predefined metrics. You can encode this as a property graph or configuration. For example:

```
# Simple semantic mapping example
business_terms = {
    "active customers": [("customers", "customer_id)], # meaning: count of
customers
    "total revenue": [("orders", "order_total)],
    "monthly churn": [("subscriptions", "cancelled"), ("subscriptions",
"signup_date)],
    # etc.
}

def map_term_to_tables(term: str):
    return business_terms.get(term.lower(), None)
```

Or use a graph library:

```
import networkx as nx
semantic_graph = nx.DiGraph()
# Nodes can be business concepts, edges link to technical columns
semantic_graph.add_node("revenue", type="concept")
semantic_graph.add_node("orders.order_total", type="column")
semantic_graph.add_edge("revenue", "orders.order_total",
relation="metric_defined_in")
```

Then queries can traverse this graph to find the relevant tables/joins. This is similar in spirit to the **dbt Semantic Layer**: MetricFlow builds a semantic graph linking models and metrics ¹⁴ ¹⁵. By encoding relationships (entities, dimensions, measures), MetricFlow “figures out the best path between tables” to compute a metric ¹⁵. A similar strategy lets the agent perform multi-hop reasoning across large schemas. For example, if the user asks for a metric defined in business terms, the agent can resolve it to specific table columns via this semantic graph.

This semantic layer helps avoid hallucinations in large schemas. It also lets you inject domain knowledge (e.g. default aggregation or filters for “active user”). For instance, if “active” customers means last 30 days, the semantic config can include that logic, which the agent applies when generating SQL.

Security, Governance, and Safety

Enforce **execution safety** as a first-class concern. For a start, confine the agent to read-only queries: block any `DROP`, `DELETE`, `UPDATE`, or other destructive commands. Implement a SQL sanitizer or validator node before execution (e.g. using regex or a SQL parser) to catch forbidden tokens. Cap returned rows (e.g. adding a default `LIMIT`) and set query timeouts to prevent runaway queries.

Guard against SQL injection and other prompt-based attacks. Recent work highlights that text-to-SQL models are vulnerable: even small injected triggers can cause them to produce malicious SQL ¹⁶. For example, one study shows backdoor attacks enabling the model to generate injected queries, underscoring severe security risks ¹⁶. In practice, ensure user inputs are sanitized and that the generated SQL is validated (e.g. using parameterized queries or whitelisting table names).

Scrub output for **PII** before returning answers. Use a simple regex-based or model-based PII detector to redact names, emails, SSNs, etc., from query results. For example, after fetching results, run a PII classification pass and replace detected PII with “[REDACTED]”. This protects privacy before summarization or display. (Newer models or tools like OpenAI’s privacy filter can also help with this task.)

Optimize **cost and latency**. Enterprise schemas can be huge. Prune irrelevant tables/columns before prompting: only include metadata for tables matching the query terms (schema linking). Use a smaller model for simple tasks (like intent routing or content retrieval) and reserve larger LLMs for final SQL generation. For example, use an open-source 7B or 13B model to interpret intent or detect errors, and call GPT-4/5 only for the complex SQL-building step. This tiered approach saves tokens and time.

SQL Generation and Self-Correction Loop

When generating SQL, combine few-shot prompting with the schema and semantic hints. Then attempt execution with the target connector. In code:

```
try:
    results = connectors[state.dialect].execute(state.sql)
    state.result = results
except Exception as e:
    state.error = str(e)
    state.needs_retry = True
```

If execution fails (syntax error, wrong table/column, etc.), enter a **correction loop**. Feed the error message back into the LLM in a new prompt: “The SQL failed with error ‘...’; revise the query.” This helps auto-correct. Distinguish between syntax errors (easy to fix) and semantic issues (e.g. empty result implies wrong filter). If repeated failures occur, escalate to user clarifications or bail with a helpful message.

This execution-feedback approach is supported by recent surveys: modern text-to-SQL systems often include an “execution-guided refinement” step, where the agent reruns generation with error context ¹⁷ ⁶. For example, many multi-agent designs separate schema linking and execution checking modules ¹⁷, which aligns with our graph design. Implementing these as LangGraph nodes ensures the agent can loop until it produces a valid, correct SQL (or decides to stop).

Evaluation and Observability

Establish a formal evaluation harness early. Use benchmarks like **Spider**, **BIRD-SQL**, and **Spider 2.0** (for enterprise workflows) to measure performance. Spider 2.0 is specifically aimed at real-world, multi-turn SQL tasks with complex schemas ¹³. Compare against baseline models: note that Spider 2.0 found code agents achieved only ~21.3% success on those tasks ¹, highlighting the difficulty. Similarly, BIRD-SQL tests correctness *and* efficiency ², so track query cost too. In addition to public benchmarks, create private tests with your own company’s data for representativeness.

Monitor metrics like **exact match accuracy**, **execution accuracy**, **clarification success rate** (how often user input clarifies vs fails), **latency**, **token usage**, and **correction frequency**. Instrument detailed logging in each LangGraph node for introspection. For example, log each SQL candidate, execution result, error message, and user clarification.

Use an observability platform such as **LangSmith** to trace the agent’s reasoning flow. LangSmith can capture every node transition, tool call, and token usage, giving “complete visibility into agent behavior” ¹⁸. Integrate LangSmith’s SDK so you can replay sessions, debug failures, and visualize where the agent spent tokens or got stuck. For example, LangSmith’s traces let you see that “intent_node” passed to “clarification_node” and so on, making debugging multi-step failures easier.

Continuously synthesize new queries (few-shot examples, SQL mutation) to retrain or fine-tune. Surveys emphasize the value of synthetic data to cover edge cases and schema changes ¹⁹. Build a pipeline that, after each deployment, uses logs of real user queries (anonymized) to generate similar QA examples and validate them, ensuring the model adapts to evolving data.

Deployment, Scaling, and Code Examples

Decide how to **deploy** and persist state. LangGraph supports durable execution: use a persistent checkpointer (database or file) to store state between interrupts ²⁰. Store secrets (DB credentials, API keys) securely (e.g. AWS Secrets Manager). Configure your connector registry via environment or config files. Add caching layers for expensive calls: e.g., cache schema metadata and frequent content queries.

Implement model routing policies: a common strategy is “small model first, large model on fail”. For instance, try a 13B model to parse simple queries; if it flags ambiguity or fails, then escalate to a 70B or GPT-4. Maintain observability dashboards (via LangSmith or Grafana) tracking error rates and costs. Before production, run red-team tests: try adversarial prompts, messy schemas, unexpected queries (like SQL injection attempts) to ensure guardrails hold.

Finally, code generation: define each LangGraph node, state schema, and connector adapter as shown above. For example, a simplified execution node might be:

```
def execute_sql_node(state: State) -> State:
    try:
        rows = connectors[state.dialect].execute(state.sql)
        state.result = rows
    except Exception as e:
        state.error = str(e)
        state.needs_retry = True
    return state

graph.add_node("execute_node", execute_sql_node)
graph.add_edge("generate_sql", "execute_node")
```

Bring it all together with a run script that:

1. Loads or initializes state (possibly from last interrupt).
2. Invokes `graph.invoke({"user_query": user_input}, config={"thread_id": session_id})`.
3. Handles interrupts by sending user responses via `Command(resume=...)`.

This completes the agent loop. In summary, our approach merges both plans: **scope & safety** → **metadata & semantic layers** → **dialect/source routing** → **LangGraph agent + HITL loop** → **SQL generation/execution/correction** → **evaluation & observability** → **deployment**. By following industry best practices (SQLAlchemy connectors, Athena/Trino for lakes, dbt’s semantic models) and academic insights (Spider 2.0, BIRD for complexity and safety), we build a robust, production-grade text-to-SQL agent ^{13 2 14 6 18}.

1 13 [2411.07763] Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows

<https://arxiv.org/abs/2411.07763>

2 BIRD-bench

<https://bird-bench.github.io/>

3 LangGraph: Agent Orchestration Framework for Reliable AI Agents

<https://www.langchain.com/langgraph>

4 5 20 Interrupts - Docs by LangChain

<https://docs.langchain.com/oss/python/langgraph/interrupts>

6 7 17 19 A Survey of Text-to-SQL in the Era of LLMs: Where are we, and where are we going?

<https://arxiv.org/html/2408.05109v5>

8 9 Engine Configuration — SQLAlchemy 2.0 Documentation

<https://docs.sqlalchemy.org/en/20/core/engines.html>

10 Interactive SQL - Serverless Query Service - Amazon Athena - AWS

<https://aws.amazon.com/athena/>

11 12 Trino | Distributed SQL query engine for big data

<https://trino.io/>

14 15 About MetricFlow | dbt Developer Hub

<https://docs.getdbt.com/docs/build/about-metricflow>

16 [2503.05445] Are Your LLM-based Text-to-SQL Models Secure? Exploring SQL Injection via Backdoor Attacks

<https://arxiv.org/abs/2503.05445>

18 LangSmith: AI Agent & LLM Observability Platform

<https://www.langchain.com/langsmith/observability>